

Proyecto de Grado: IA al servicio de las comunidades

Maestría en Analítica para la Inteligencia de Negocios

Nombres: María Fernanda Izquierdo Aparicio & Ana María Ochoa Muñoz

Fecha: 26 de Abril de 2025

Este modelo forma parte del proyecto de grado de la Maestría en Analítica para la Inteligencia de Negocios de la Pontificia Universidad Javeriana, enfocado en la aplicación de Inteligencia Artificial al servicio de las comunidades. Su objetivo es clasificar imágenes de plantas en función de su especie usando una arquitectura pre-entrenada RESNET

1. Importe de Librerías

```
In [3]: import json

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

import os
import zipfile
import shutil
import seaborn as sns
import datetime
import cv2

import keras
from keras import models
from keras import layers

import random

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras import Layer
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.preprocessing import image
```

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.layers import GlobalAveragePooling2D
from tensorflow.keras.utils import to_categorical

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score
from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrixDisplay
from sklearn.metrics import roc_curve, auc
from sklearn.metrics import classification_report

from PIL import Image # Librería para abrir y manipular imágenes

# ignoring warnings
import warnings
warnings.simplefilter("ignore")

```

```

In [4]: #Medir_tiempo_ejecucion():
        inicio = datetime.datetime.now()
        print(f"Inicio: {inicio.strftime('%Y-%m-%d %H:%M:%S.%f')}")

```

Inicio: 2025-04-29 07:16:24.572197

2. Definición de Rutas y Parámetros

```

In [6]: data_dir = 'Datos Finales' # Reemplaza con la ruta real
        metadata_file = 'Metadatos_Finales.csv' # Reemplaza con la ruta real
        image_height = 150
        image_width = 150
        batch_size = 15
        num_classes = None # Se determinará automáticamente del archivo CSV

```

3. Cargar y procesar los metadatos

```

In [8]: try:
        metadata = pd.read_csv(metadata_file, sep=";")
        metadata["Planta_Condición_Enfermedad"] = metadata["Planta_Estado"] + "_" + metadata["Planta_Condición_Enfermedad"]
        metadata = metadata[metadata["Planta"] != "Orange"]
        metadata = metadata[metadata["Planta"] != "Grape"]
        metadata = metadata[metadata["Planta"] != "Pepper_bell"]
        metadata = metadata[metadata["Planta"] != "Peach"]
        metadata = metadata.rename(columns={'Nombre del archivo': 'image_id'})

```

```

# Asegúrate de que las columnas 'filename' (o el nombre de tu columna de
# y 'label' (o el nombre de tu columna de etiquetas) existan.

if 'image_id' not in metadata.columns or 'Planta_Condición_Enfermedad' not in metadata.columns:
    raise ValueError("El archivo CSV debe contener las columnas 'image_id' y 'Planta_Condición_Enfermedad'")
class_names = metadata['Planta_Condición_Enfermedad'].unique().tolist()
num_classes = len(class_names)
print(f"Se encontraron {num_classes} clases: {class_names}")
except FileNotFoundError:
    print(f"Error: No se encontró el archivo de metadatos en '{metadata_file}'")
    exit()
except ValueError as e:
    print(f"Error al procesar el archivo CSV: {e}")
    exit()

```

Se encontraron 37 clases: ['Yuca_sick_Cassava Bacterial Blight (CBB)', 'Yuca_sick_Cassava Mosaic Disease (CMD)', 'Yuca_sick_Cassava Brown Streak Disease (CBSD)', 'Yuca_sick_Cassava Green Mottle (CGM)', 'Yuca_healthy_healthy', 'Strawberry_healthy_healthy', 'Potato_sick_Early_blight', 'Corn_(maize)_healthy_healthy', 'Tomato_sick_Target_Spot', 'Potato_sick_Late_blight', 'Tomato_sick_Late_blight', 'Tomato_sick_Tomato_mosaic_virus', 'Tomato_sick_Leaf_Mold', 'Apple_sick_Cedar_apple_rust', 'Tomato_sick_Bacterial_spot', 'Tomato_sick_Early_blight', 'Corn_(maize)_sick_Common_rust', 'Tomato_healthy_healthy', 'Tomato_sick_Tomato_Yellow_Leaf_Curl_Virus', 'Apple_sick_Apple_scab', 'Corn_(maize)_sick_Northern_Leaf_Blight', 'Tomato_sick_Spider_mites Two-spotted_spider_mite', 'Tomato_sick_Septoria_leaf_spot', 'Corn_(maize)_sick_Cercospora_leaf_spot Gray_leaf_spot', 'Apple_sick_Black_rot', 'Apple_healthy_healthy', 'Strawberry_sick_Leaf_scorch', 'Potato_healthy_healthy', 'cauliflower_sick_downy_mildew', 'cucumber_healthy_healthy', 'bean_sick_angular_leaf_spot', 'cauliflower_healthy_healthy', 'cauliflower_sick_black_rot', 'bean_sick_rust', 'cauliflower_sick_bacterial_spot_rot', 'bean_healthy_healthy', 'cucumber_sick_diseased']

```
In [9]: train, test = train_test_split(metadata, test_size=0.2, random_state=42, shuffle=True)
        train.shape, test.shape
```

```
Out[9]: ((42354, 8), (10589, 8))
```

4. Crear un generador de datos

```
In [11]: datagen = ImageDataGenerator(
        validation_split = 0.2,
        preprocessing_function = None,
        rotation_range = 45,
        zoom_range = 0.2,
        horizontal_flip = True,
        vertical_flip = True,
        fill_mode = 'nearest',
    )
```

```

        shear_range = 0.1,
        height_shift_range = 0.1,
        width_shift_range = 0.1
    )

    train_generator = datagen.flow_from_dataframe(
        dataframe=train,
        directory=data_dir,
        x_col='image_id',
        y_col='Planta Condición Enfermedad',
        target_size=(image_height, image_width),
        batch_size=batch_size,
        class_mode='categorical', # Importante para clasificación multiclase
        subset='training',
        seed=42,
        classes=class_names # Asegurarse del orden de las clases
    )

    validation_generator = datagen.flow_from_dataframe(
        dataframe=train,
        directory=data_dir,
        x_col='image_id',
        y_col='Planta Condición Enfermedad',
        target_size=(image_height, image_width),
        batch_size=batch_size,
        class_mode='categorical',
        subset='validation',
        seed=42,
        classes=class_names
    )

```

Found 33880 validated image filenames belonging to 37 classes.
 Found 8470 validated image filenames belonging to 37 classes.

5. Cargar el modelo ResNet50 preentrenado (sin la capa superior de clasificación)

```
In [13]: base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(in
```

6. Congelar las capas convolucionales base

```
In [15]: for layer in base_model.layers:
        layer.trainable = False
```

7. Añadir una nueva capa de clasificación

personalizada

```
In [17]: x = base_model.output
x = GlobalAveragePooling2D()(x)
# Puedes añadir más capas densas aquí si lo deseas
# x = Dense(1024, activation='relu')(x)
predictions = Dense(num_classes, activation='softmax')(x)
```

8. Crear el modelo final

```
In [19]: model = Model(inputs=base_model.input, outputs=predictions)
```

9. Compilar el modelo para fine-tuning de la última capa

```
In [21]: model.compile(optimizer='adam',
                        loss='categorical_crossentropy',
                        metrics=['accuracy'])
```

10. Entrenar el modelo con la última capa descongelada

```
In [23]: epochs_fine_tune = 20 # Define el número de épocas para el fine-tuning
```

```
In [24]: print("Comenzando el entrenamiento de la última capa...")
history_fine_tune = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // train_generator.batch_size,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // validation_generator.batch_size,
    epochs=epochs_fine_tune,
    verbose = 1
)
```

Comenzando el entrenamiento de la última capa...

Epoch 1/20

2258/2258 ————— 720s 318ms/step - accuracy: 0.6547 - loss: 1.1820 - val_accuracy: 0.7805 - val_loss: 0.6644

Epoch 2/20

2258/2258 ————— 154s 68ms/step - accuracy: 0.6667 - loss: 0.8453 - val_accuracy: 0.7747 - val_loss: 0.6652

Epoch 3/20

2258/2258 ————— 781s 346ms/step - accuracy: 0.7854 - loss: 0.

6622 - val_accuracy: 0.7865 - val_loss: 0.6828
Epoch 4/20
2258/2258 ————— **158s** 70ms/step - accuracy: 0.8667 - loss: 0.5
562 - val_accuracy: 0.7865 - val_loss: 0.6852
Epoch 5/20
2258/2258 ————— **798s** 354ms/step - accuracy: 0.7918 - loss: 0.
6378 - val_accuracy: 0.7890 - val_loss: 0.6426
Epoch 6/20
2258/2258 ————— **158s** 70ms/step - accuracy: 0.7333 - loss: 0.7
970 - val_accuracy: 0.7765 - val_loss: 0.6681
Epoch 7/20
2258/2258 ————— **794s** 351ms/step - accuracy: 0.8016 - loss: 0.
6155 - val_accuracy: 0.8082 - val_loss: 0.6326
Epoch 8/20
2258/2258 ————— **158s** 70ms/step - accuracy: 0.9333 - loss: 0.3
197 - val_accuracy: 0.8069 - val_loss: 0.6297
Epoch 9/20
2258/2258 ————— **805s** 356ms/step - accuracy: 0.8160 - loss: 0.
5784 - val_accuracy: 0.8035 - val_loss: 0.6240
Epoch 10/20
2258/2258 ————— **162s** 72ms/step - accuracy: 0.8667 - loss: 0.3
896 - val_accuracy: 0.8051 - val_loss: 0.6274
Epoch 11/20
2258/2258 ————— **821s** 364ms/step - accuracy: 0.8178 - loss: 0.
5714 - val_accuracy: 0.7805 - val_loss: 0.6574
Epoch 12/20
2258/2258 ————— **164s** 73ms/step - accuracy: 0.8667 - loss: 0.5
173 - val_accuracy: 0.7876 - val_loss: 0.6517
Epoch 13/20
2258/2258 ————— **850s** 377ms/step - accuracy: 0.8163 - loss: 0.
5742 - val_accuracy: 0.8073 - val_loss: 0.6727
Epoch 14/20
2258/2258 ————— **166s** 74ms/step - accuracy: 0.8000 - loss: 0.6
180 - val_accuracy: 0.8132 - val_loss: 0.6615
Epoch 15/20
2258/2258 ————— **868s** 384ms/step - accuracy: 0.8162 - loss: 0.
5709 - val_accuracy: 0.8165 - val_loss: 0.6469
Epoch 16/20
2258/2258 ————— **179s** 79ms/step - accuracy: 0.7333 - loss: 1.4
707 - val_accuracy: 0.8098 - val_loss: 0.6503
Epoch 17/20
2258/2258 ————— **957s** 424ms/step - accuracy: 0.8212 - loss: 0.
5662 - val_accuracy: 0.8072 - val_loss: 0.6205
Epoch 18/20
2258/2258 ————— **198s** 88ms/step - accuracy: 0.9333 - loss: 0.1
758 - val_accuracy: 0.8050 - val_loss: 0.6348
Epoch 19/20
2258/2258 ————— **953s** 422ms/step - accuracy: 0.8223 - loss: 0.
5618 - val_accuracy: 0.8227 - val_loss: 0.6022
Epoch 20/20

2258/2258 ————— 181s 80ms/step – accuracy: 0.8667 – loss: 0.4043 – val_accuracy: 0.8228 – val_loss: 0.5965

```
In [25]: fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

Fin: 2025-04-29 10:03:31.684677
Duración: 2:47:07.112480

11. Opcional: Descongelar algunas de las últimas capas del modelo base para un fine-tuning más profundo

```
In [27]: # unfreeze_layers = 5 # Número de capas a descongelar (contando desde la última
# for layer in base_model.layers[-unfreeze_layers:]:
#     layer.trainable = True
```

12. Opcional: Recompilar el modelo después de descongelar las capas

```
In [29]: #model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5), # Us
#         loss='categorical_crossentropy',
#         metrics=['accuracy'])
```

13. Opcional: Entrenar el modelo con las capas descongeladas

```
In [31]: #epochs_deep_fine_tune = 5 # Define el número de épocas para el fine-tuning

#print(f"\nComenzando el fine-tuning profundo de las últimas {unfreeze_layers} capas")
#history_deep_fine_tune = model.fit(
#    train_generator,
#    steps_per_epoch=train_generator.samples // train_generator.batch_size,
#    validation_data=validation_generator,
#    validation_steps=validation_generator.samples // validation_generator.batch_size,
#    epochs=epochs_deep_fine_tune
#)
```

14. Evaluar el modelo final

```
In [33]: print("\nEvaluando el modelo...")
evaluation_result = model.evaluate(validation_generator)
print(f"Pérdida en la validación: {evaluation_result[0]:.4f}")
print(f"Precisión en la validación: {evaluation_result[1]:.4f}")
```

Evaluando el modelo...

565/565  **182s** 322ms/step - accuracy: 0.8170 - loss: 0.6086

Pérdida en la validación: 0.6031

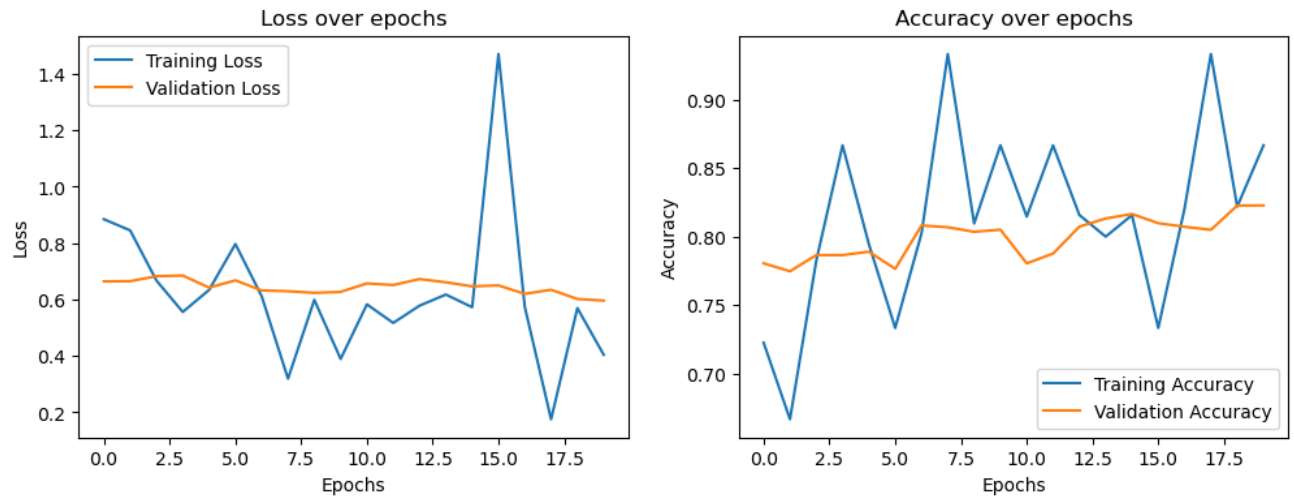
Precisión en la validación: 0.8152

```
In [34]: # Acceder a los datos de pérdida y precisión
loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
accuracy = history_fine_tune.history['accuracy']
val_accuracy = history_fine_tune.history['val_accuracy']

# Graficar la pérdida del modelo
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history_fine_tune.history['loss'], label='Training Loss')
plt.plot(history_fine_tune.history['val_loss'], label='Validation Loss')
plt.title('Loss over epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

# Graficar la precisión del modelo
plt.subplot(1, 2, 2)
plt.plot(history_fine_tune.history['accuracy'], label='Training Accuracy')
plt.plot(history_fine_tune.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy over epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
```

Out[34]: <matplotlib.legend.Legend at 0x287567200>



```
In [35]: epochs = list(range(1, 21))
data = {
    'Epoch': epochs,
    'Training Loss': loss,
    'Validation Loss': val_loss,
    'Training Accuracy': accuracy,
    'Validation Accuracy': val_accuracy
}
df = pd.DataFrame(data)

# Imprimir la tabla
print(df)
```

	Epoch	Trainning Loss	Validation Loss	Training Accuracy \
0	1	0.885015	0.664414	0.722516
1	2	0.845317	0.665204	0.666667
2	3	0.667475	0.682815	0.782519
3	4	0.556211	0.685186	0.866667
4	5	0.634290	0.642644	0.794508
5	6	0.797002	0.668142	0.733333
6	7	0.611808	0.632560	0.802392
7	8	0.319707	0.629709	0.933333
8	9	0.599120	0.623996	0.809597
9	10	0.389632	0.627385	0.866667
10	11	0.583073	0.657397	0.814676
11	12	0.517342	0.651720	0.866667
12	13	0.578514	0.672735	0.815798
13	14	0.617956	0.661454	0.800000
14	15	0.573014	0.646893	0.815709
15	16	1.470665	0.650283	0.733333
16	17	0.574282	0.620492	0.820670
17	18	0.175793	0.634752	0.933333
18	19	0.569781	0.602162	0.821940
19	20	0.404319	0.596545	0.866667

	Validation Accuracy
0	0.780496
1	0.774705
2	0.786525
3	0.786525
4	0.789007
5	0.776478
6	0.808156
7	0.806856
8	0.803546
9	0.805083
10	0.780496
11	0.787589
12	0.807329
13	0.813239
14	0.816548
15	0.809811
16	0.807210
17	0.804965
18	0.822695
19	0.822813

```
In [36]: # Accuracy final en entrenamiento
val_loss = history_fine_tune.history['val_loss'][-1]
```

```
In [37]: print('Pérdida final en Train:', history_fine_tune.history['loss'][-1])
print("Pérdida final en Validation:", history_fine_tune.history['val_loss'][-1])
```

```
print("Accuracy final en Train:", history_fine_tune.history['accuracy'][-1])
print("Accuracy final en Validation:", history_fine_tune.history['val_accuracy'][-1])
```

Perdida final en Train: 0.40431949496269226
 Pérdida final en Validation: 0.5965449810028076
 Accuracy final en Train: 0.8666666746139526
 Accuracy final en Validation: 0.8228132128715515

```
In [46]: class_indices = train_generator.class_indices # o test_generator.class_indices
index_to_class = {v: k for k, v in class_indices.items()}
print(index_to_class)
with open("index_to_class_modelo11.json", "w") as f:
    json.dump(index_to_class, f, indent=4, ensure_ascii=False)
```

```
{0: 'Yuca_sick_Cassava Bacterial Blight (CBB)', 1: 'Yuca_sick_Cassava Mosaic Disease (CMD)', 2: 'Yuca_sick_Cassava Brown Streak Disease (CBSD)', 3: 'Yuca_sick_Cassava Green Mottle (CGM)', 4: 'Yuca_healthy_healthy', 5: 'Strawberry_healthy_healthy', 6: 'Potato_sick_Early_blight', 7: 'Corn_(maize)_healthy_healthy', 8: 'Tomato_sick_Target_Spot', 9: 'Potato_sick_Late_blight', 10: 'Tomato_sick_Late_blight', 11: 'Tomato_sick_Tomato_mosaic_virus', 12: 'Tomato_sick_Leaf_Mold', 13: 'Apple_sick_Cedar_apple_rust', 14: 'Tomato_sick_Bacterial_spot', 15: 'Tomato_sick_Early_blight', 16: 'Corn_(maize)_sick_Common_rust', 17: 'Tomato_healthy_healthy', 18: 'Tomato_sick_Tomato_Yellow_Leaf_Curl_Virus', 19: 'Apple_sick_Apple_scab', 20: 'Corn_(maize)_sick_Northern_Leaf_Blight', 21: 'Tomato_sick_Spider_mites Two-spotted_spider_mite', 22: 'Tomato_sick_Septoria_leaf_spot', 23: 'Corn_(maize)_sick_Cercospora_leaf_spot Gray_leaf_spot', 24: 'Apple_sick_Black_rot', 25: 'Apple_healthy_healthy', 26: 'Strawberry_sick_Leaf_scorch', 27: 'Potato_healthy_healthy', 28: 'cauliflower_sick_downy_mildew', 29: 'cucumber_healthy_healthy', 30: 'bean_sick_angular_leaf_spot', 31: 'cauliflower_healthy_healthy', 32: 'cauliflower_sick_black_rot', 33: 'bean_sick_rust', 34: 'cauliflower_sick_bacterial_spot_rot', 35: 'bean_healthy_healthy', 36: 'cucumber_sick_diseased'}
```

15. Testeo del Modelo

```
In [49]: test.Planta = test.Planta.astype('str')

test_generator = datagen.flow_from_dataframe(test,
    directory = "Datos Finales",
    subset = "training",
    x_col = "image_id",
    y_col = "Planta_Condición_Enfermedad",
    target_size = (image_height, image_width),
    batch_size = batch_size,
    class_mode = "categorical")
```

Found 8472 validated image filenames belonging to 37 classes.

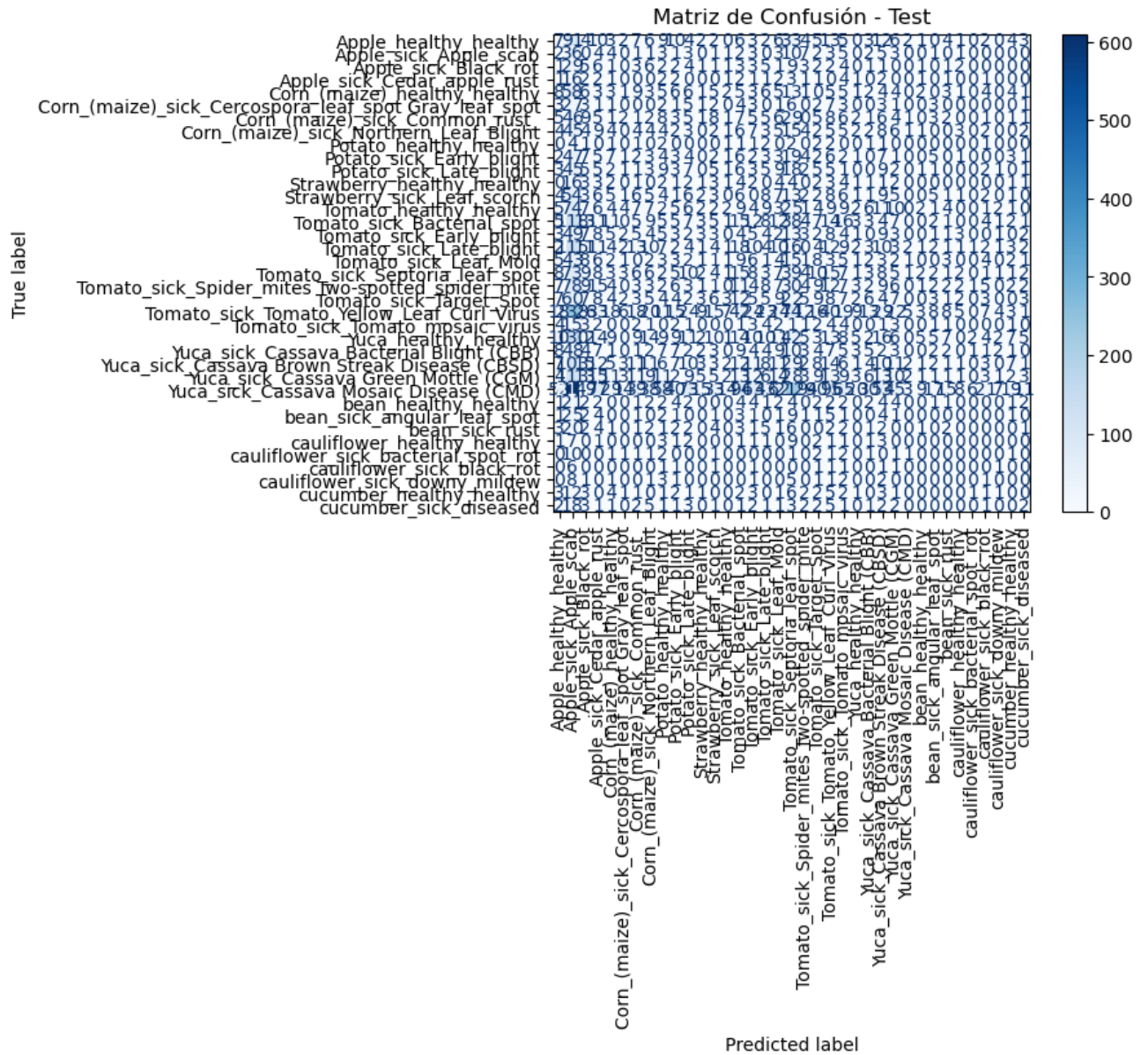
```
In [50]: # -----
```

```
# 1. Obtener predicciones completas para test
# -----
y_pred_probs = model.predict(test_generator)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = test_generator.classes
```

565/565 ————— 148s 260ms/step

```
In [51]: # -----
# 2. Matriz de Confusión
# -----
plt.figure(figsize=(200, 80))
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=test_gener
disp.plot(cmap="Blues", xticks_rotation=90)
plt.title("Matriz de Confusión - Test")
plt.savefig("matriz_confusion_test_modelo_11.png", dpi=300, bbox_inches='tight')
plt.show()
```

<Figure size 20000x8000 with 0 Axes>



```
In [52]: accuracy = accuracy_score(y_true, y_pred)
print(f"Test Accuracy: {accuracy:.4f}")
```

Test Accuracy: 0.0280

```
In [53]: # Función para normalizar imágenes solo para visualización
def normalize_for_display(img):
    img_min = img.min()
    img_max = img.max()
    if img_max > 1.0 or img_min < 0.0:
        img = (img - img_min) / (img_max - img_min + 1e-7)
    return img

# Obtener nombres de clases
class_indices = test_generator.class_indices
class_names = list(class_indices.keys())
```

```
# Reiniciar el generador para empezar desde el principio
test_generator.reset()
x_batch, y_true_batch = next(test_generator)

# Obtener predicciones del modelo
y_pred_probs_batch = model.predict(x_batch)
y_pred_batch = np.argmax(y_pred_probs_batch, axis=1)
y_true_batch = np.argmax(y_true_batch, axis=1)

# Número de imágenes a mostrar (máximo el tamaño del batch)
num_samples = x_batch.shape[0]

# Mostrar imágenes con etiquetas
plt.figure(figsize=(15, 8))
for i in range(num_samples):
    plt.subplot(3, 4, i + 1)
    img_to_show = normalize_for_display(x_batch[i])
    plt.imshow(img_to_show)
    plt.axis('off')
    plt.title(
        f"Real: {class_names[y_true_batch[i]]}\nPred: {class_names[y_pred_batch[i]]}
        color='green' if y_true_batch[i] == y_pred_batch[i] else 'red'
    )
plt.tight_layout()
plt.suptitle("Muestreo de Predicciones vs. Etiquetas Reales", fontsize=16, y=0.95)
plt.show()
```

1/1 ————— 1s 768ms/step

```

-----
ValueError                                Traceback (most recent call last)
Cell In[53], line 28
    26 plt.figure(figsize=(15, 8))
    27 for i in range(num_samples):
--> 28     plt.subplot(3, 4, i + 1)
    29     img_to_show = normalize_for_display(x_batch[i])
    30     plt.imshow(img_to_show)

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/pyplot.py:1534,
in subplot(*args, **kwargs)
    1531 fig = gcf()
    1533 # First, search for an existing subplot with a matching spec.
-> 1534 key = SubplotSpec._from_subplot_args(fig, args)
    1536 for ax in fig.axes:
    1537     # If we found an Axes at the position, we can reuse it if the us
er passed no
    1538     # kwargs or if the Axes class and kwargs are identical.
    1539     if (ax.get_subplotspec() == key
    1540         and (kwargs == {}
    1541             or (ax._projection_init
    1542                 == fig._process_projection_requirements(**kwargs)))
    ):

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/gridspec.py:589,
in SubplotSpec._from_subplot_args(figure, args)
    587 else:
    588     if not isinstance(num, Integral) or num < 1 or num > rows*cols:
--> 589         raise ValueError(
    590             f"num must be an integer with 1 <= num <= {rows*cols}, "
    591             f"not {num!r}"
    592         )
    593     i = j = num
    594 return gs[i-1:j]

ValueError: num must be an integer with 1 <= num <= 12, not 13

```




```
In [ ]: # 3. Convertir etiquetas reales a one-hot
n_classes = y_pred_probs.shape[1]
y_true_onehot = to_categorical(y_true, num_classes=n_classes)

# 4. Calcular ROC y AUC por clase
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_onehot[:, i], y_pred_probs[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# 5. Obtener nombres de clase
class_names = list(test_generator.class_indices.keys())

# 6. Graficar curvas ROC
plt.figure(figsize=(12, 8))
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i], label=f'{class_names[i]} (AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Aleatorio')
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos')
plt.title('Curvas ROC por clase')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```

```
In [ ]: from sklearn.metrics import roc_auc_score
```



```
# AUC macro (promedio de clases, todas igual de importantes)
auc_macro = roc_auc_score(y_true_onehot, y_pred_probs, average='macro')

# AUC micro (promedia sobre todas las instancias)
auc_micro = roc_auc_score(y_true_onehot, y_pred_probs, average='micro')

print(f"AUC Macro promedio: {auc_macro:.4f}")
print(f"AUC Micro promedio: {auc_micro:.4f}")
```

```
In [ ]: # Asegúrate de tener esto:
# y_pred = np.argmax(y_pred_probs, axis=1)
# y_true = test_generator.classes

# Accuracy
accuracy = accuracy_score(y_true, y_pred)

# F1 Scores
f1_macro = f1_score(y_true, y_pred, average='macro')
f1_micro = f1_score(y_true, y_pred, average='micro')
f1_weighted = f1_score(y_true, y_pred, average='weighted')

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 Score (macro): {f1_macro:.4f}")
print(f"F1 Score (micro): {f1_micro:.4f}")
print(f"F1 Score (weighted): {f1_weighted:.4f}")

# Detalle por clase
class_names = list(test_generator.class_indices.keys())
print("\nReporte de clasificación por clase:")
print(classification_report(y_true, y_pred, target_names=class_names))
```

16. Guardar el modelo entrenado

```
In [ ]: model.save('modelo_11_resnet50_fine_tuned.h5')
```

```
In [ ]: class_indices = train_generator.class_indices # o test_generator.class_indices
index_to_class = {v: k for k, v in class_indices.items()}
```

```
In [ ]:
```